

CONFIDENTIAL

Analistas de seguridad:

Kleber Leonardo de Carvalho

Luigui Juan Guillermo Ramos

Paula Benítez

Cliente / Entidad FinTech SecBank S.A.

Data de Emissão Outubro de 2026

Clasificación del documento: confidencial / uso interno

Control del documento	Detalle
Entregable	**Informe técnico de IntegriDos para FinTech SecBank S.A**
Tecnología seleccionada	**HTTP/1.1 con endpoints de API de estilo REST (implementación ligera; no R9EST completo)**
Estado de evidencias	**Kali Linux, loopback 127.0.0.1, HTTP; los registros muestran Python 3.14.7 y Wireshark 4.6.4 , Burpsuit**
Restricción del ejercicio	**No utilizar TLS/HTTPS**

SEVILLA -SPN

CONFIDENTIAL

DOCUMENTACIÓN TÉCNICA OFICIAL: PROYECTO (SECBANK)

1. RESUMEN EJECUTIVO

La entidad financiera SecBank opera en un modelo distribuido cliente-servidor enfocado en el ecosistema FinTech, procesando operaciones sensibles como la autenticación de usuarios y órdenes de transferencia bancaria a través de redes públicas no seguras (como Internet). En una arquitectura de esta naturaleza, el tráfico de datos se encuentra expuesto a vectores de ataque. Se revisó el prototipo modelo para valorar sus controles de integridad, autenticidad y protección contra repetición. El código implementa HMAC-SHA256 sobre el cuerpo, el nonce y el timestamp; rechaza nonces reutilizados y timestamps fuera de una ventana de 60 segundos; deriva contraseñas con PBKDF2-HMAC-SHA256, salt aleatorio de 16 bytes y 600.000 iteraciones; y utiliza `hmac.compare_digest()` para comparar MAC y hashes de contraseña.

Las evidencias disponibles muestran resultados reales en el laboratorio: registro, inicio de sesión y transferencia legítimos con HTTP 200; alteración de una transferencia interceptada con respuesta HTTP 403; reenvío de una transferencia aceptada con respuesta HTTP 403; tres intentos de login incorrectos con HTTP 401 seguidos por bloqueo HTTP 429; y cierre de sesión con HTTP 200. El benchmark local registró dos ejecuciones de diez mediciones por caso: medianas de 0,694 ms frente a 0,704 ms y, en otra ejecución, 1,011 ms frente a 0,915 ms.

1.1 Recomendación a SecBank:

En estricto cumplimiento con la Política de Seguridad de SecBank y Scopo, este proyecto adopta: la prohibición de seguridad en la capa de transporte (tales como TLS, HTTPS o SSL). No está preparada para un piloto financiero ni para procesar información real. adecuado únicamente como demostrador aislado. No se debe procesar fondos, credenciales reales ni datos de clientes. HTTP transmite contenido sin cifrado; la firma no cubre método, ruta ni cookie de sesión; y el secreto HMAC reside en un archivo accesible a los componentes cliente y servidor. `hmac.compare_digest()` es una buena mitigación para la comparación del MAC, pero el benchmark extremo a extremo no demuestra que todo el sistema sea de tiempo constante ni que no existan otros canales laterales.

2. DISEÑO DEL PROTOCOLO

Para garantizar la total transparencia y reproducibilidad del sistema, el protocolo de comunicación de SecBank se rige.: El cliente usa HTTP/1.1 con solicitudes `POST` a `/signin`, `/login`, `/transfer` y `/logout`. El servidor escucha en `127.0.0.1` y, por defecto,

CONFIDENTIAL

en el puerto 8080; para las pruebas se configuró el puerto 8081 y el proxy Burp en 8082. Registro e inicio de sesión envían datos de formulario; la transferencia envía JSON con `tx_id`, cuentas, importe, moneda y timestamp.

Para cada solicitud, el cliente genera un *nonce* UUID y una marca de tiempo Unix. La firma se calcula mediante HMAC-SHA256 a partir de los bytes del cuerpo concatenados con el *nonce* y la marca de tiempo en formato de texto. El servidor verifica que la marca de tiempo sea válida y no difiera más de 60 segundos del reloj local, que el *nonce* no se haya utilizado previamente y que el HMAC coincida. Tras procesar la ruta, en los casos de transferencia y cierre de sesión, la *cookie* de sesión se valida por separado. El registro de transacciones y *nonces* se mantiene en formato JSON, mientras que las sesiones activas se almacenan en memoria.

2.1 DESCRIPCIÓN DETALLADA DE LOS PASOS DEL PROTOCOLO

Secuencia de operación legítima

```
Cliente                          Servidor
|                                |
|-- POST /login + cuerpo + nonce, ts -->| comprobar HMAC, ventana y nonce
|                                | verificar contraseña (PBKDF2)
|<-- 200 + Set-Cookie session_id -------| crear sesión en memoria
|                                |
|-- POST /transfer + JSON + HMAC ------>| validate HMAC, nonce, sesión y campos
|                                | registrar tx_id en JSON
|<-- 200 Transacción exitosa ----------|
|                                |
|-- POST /logout + HMAC + cookie ------>| invalidar sesión
|<-- 200 + expiración de cookie --------|
```

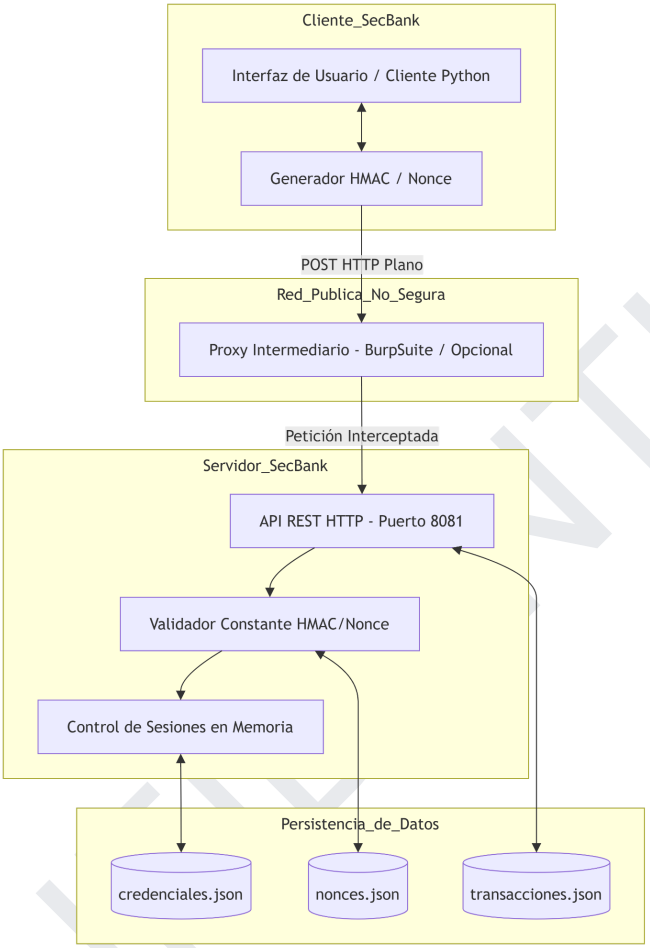
Secuencia de manipulación y replay

```
Cliente          Burp (intermediario)          Servidor
|-- transferencia firmada -->|---------------------->|
|                    |     respuesta 200   |
|                    |                     |
|-- nueva transferencia ---->| cambia amount;       |
|                    | conserva la firma --->| HMAC no coincide
|<-------------------------|<------ HTTP 403 ------|
|                    |                     |
|                    | envía solicitud     |
|                    | previamente aceptada->| nonce repetido
```

CONFIDENTIAL

La firma auténtica solamente cuerpo, nonce y timestamp; no vincula criptográficamente método HTTP, ruta, cookie ni otros encabezados. Por eso no debe interpretarse como autenticación integral de la solicitud.

2.2 Diagrama visual de arquitectura



3. ANÁLISIS DE SEGURIDAD Y EVIDENCIAS

Prueba	Evidencia revisada	Resultado observado
Flujo norma	evidencias/1normal.pcapng log1_cliente.txt log1_servidor_transcrito.txt	Registro duplicado rechazado (403); registro de usuario, login y transferencia de prueba aceptados (200)
Manipulación MitM	evidencias/2mitm.pcapng	En una transferencia capturada, el cuerpo se modificó (importe original 10,0; modificado 99999,0) y se mantuvieron los encabezados de autenticación. El servidor respondió 403. El PCAP contiene datos HTTP en claro: no distribuir sin redactar.
Replay	evidencias/3replay.pcapng log3_burp_historico_transcrito.txt	Una transferencia recibió 200; el reenvío de la solicitud fue rechazado

CONFIDENTIAL

		con 403. Compatible con la detección de nonce reutilizado.
Bloqueo de login	log5_cliente.txt log5_servidor_transcrito.txt evidencias/forçabruta.png	Tres contraseñas incorrectas recibieron 401; el intento posterior con la contraseña correcta recibió 429 por bloqueo temporal.
Benchmark	log7_benchmark_saida.txt	Diez solicitudes por caso en cada ejecución. Medianas: 0,694/0,704 ms y 1,011/0,915 ms (error al inicio/final de la firma). Todas las solicitudes inválidas deben responder 403.
Logout	evidencias/5logoutexit.pcapng transcripciones log6_*	Cierre de sesión con HTTP 200; la transcripción posterior registra rechazo 403 de una transferencia sin sesión activa.

3.2 Interpretación, límites y recomendaciones

Control	Resultado observado	Límite de la evidencia
Integridad frente a modificación	La captura MitM demuestra que alterar el cuerpo firmado sin recalcular el HMAC provoca el rechazo de la solicitud.	La firma no cifra el tráfico.
Antirreplay	El reenvío observado fue rechazado	El servidor almacena nonces en una lista JSON, sin caducidad o purga visible. No se presenta una prueba de concurrencia
Comparación del MAC y de contraseñas	hasher.py usa hmac.compare_digest() para el MAC y el hash de contraseña, evitando la comparación ordinaria que se detiene en el primer carácter distinto	Esto no hace que el endpoint completo tenga tiempo constante. El benchmark mide HTTP extremo a extremo con solo diez observaciones por caso; las diferencias pequeñas observadas no prueban ni refutan por sí mismas la ausencia de un canal lateral.

Evolución del modelo de firmado (Binding Criptográfico): Se observó que la firma HMAC actual valida rigurosamente el cuerpo JSON, el nonce y el timestamp. No obstante, al no incluir la ruta HTTP (ej. /transfer) ni la cookie de sesión (session_id) en la cadena de bytes firmada, existe un vector teórico donde una carga útil válida podría reenviarse a un endpoint distinto. La mitigación ideal, identificada por el equipo pero fuera del alcance del actual laboratorio, consistiría en ampliar la base de la firma. En lugar de concatenar solo datos, el mensaje firmado debería estructurarse como: Ruta_HTTP + b"|" + Cookie_ID + b"|" + Datos + b"|" + Nonce + b"|" + Timestamp.

CONFIDENTIAL

4. MANUAL DE DESPLIEGUE Y EJECUCIÓN

El procedimiento evidenciado usa Kali/Linux y Python 3, desde la carpeta del proyecto. `secret.key` debe existir y ser el mismo para cliente y servidor; `hasher.py` crea una clave aleatoria de 32 bytes si no existe. No copiarla al informe.

Terminal 1 — iniciar el servidor en el puerto usado por las pruebas

export SECBANK_SERVER_PORT=8081

python3 Servidor.py

Terminal 2 — iniciar el cliente directamente

export SECBANK_SERVER_PORT=8081

python3 Cliente.py

Para observar el tráfico por Burp, configurar un listener HTTP en `127.0.0.1:8082` y, en el Terminal 2, añadir antes de ejecutar el cliente:

export SECBANK_BURP_PROXY=http://127.0.0.1:8082

En el cliente, usar `signin`, `login`, `transfer`, `logout` y `exit`. Para las pruebas auxiliares, con el servidor activo y en otro terminal:

export SECBANK_SERVER_PORT=8081

python3 3-ataque_fuerza_bruta.py

python3 4-benchmark_timing.py

El primer script crea un usuario ficticio y lo bloquea temporalmente; el segundo envía firmas inválidas. En Wireshark, seleccionar loopback y usar el filtro `tcp.port == 8081 or tcp.port == 8082`. Las pruebas manuales MitM y replay se realizan en Burp y Wireshark.

Gestión de Memoria y Purga de Nonces.: El equipo identificó que el sistema registra los nonces utilizados en nonces.json de forma acumulativa sin una rutina de limpieza. A escala comercial, esto generaría un desbordamiento de almacenamiento. Sin embargo, dado que el servidor impone una ventana temporal estricta de 60 segundos basada en el timestamp, cualquier repetición antigua es rechazada independientemente del registro de nonces. Para este prototipo demostrador, la acumulación es aceptable, pero en producción se recomendaría implementar un cronjob o tarea en segundo plano que purgue los nonces expirados periódicamente.

CONFIDENTIAL

5. MATRIZ DE TRAZABILIDAD DE REQUISITOS

Requisito	Implementación	Evidencia/observación
RF1 — Registro de usuario y rechazo de duplicados	Cliente.py:84–104 Servidor.py:90–100	log1_* registra duplicado 403 y registro 200.
RF1 — Login y gestión de sesión/logout	Cliente.py:107–140, 183–207 Servidor.py:17, 102–143	Login y logout 200; sesión en memoria. No hay expiración automática observada.
RF2 — Transferencia JSON y registro	Cliente.py:142–174 Servidor.py:147–188	Transferencia normal 200; validación de negocio limitada.
RS1 — Hash de contraseña con salt	hasher.py:25–39, 53–65	PBKDF2-HMAC-SHA256, salt aleatorio de 16 bytes, 600.000 iteraciones; comparación con compare_digest.
RS1 — Bloqueo tras intentos fallidos	hasher.py:21–22, 41–70 Servidor.py:102–124 3-ataque_fuerza_bruta.py:55–88	3 respuestas 401 seguidas por 429; bloqueo de 300 segundos.
RS2 — Firma HMAC-SHA256	hasher.py:75–84 Cliente.py:96–103, 120–127, 159–168 Servidor.py:67–86	MAC cubre cuerpo + nonce + timestamp; no cubre ruta, método ni cookie.
RS2 — Clave aleatoria de al menos 256 bits	hasher.py:10–18	32 bytes mediante secrets; sin ciclo de gestión/rotación en el prototipo.
RS3 — Timestamp y ventana de frescura	Cliente.py:96–103 Servidor.py:71–86	Tolerancia de 60 segundos.
RS3 — Nonce único / bloqueo de replay	Cliente.py:96–103 Servidor.py:21–25, 77–88	Replay observado en 3replay.pcapng: 200 inicial y 403 al reenvío. Persistencia JSON sin purga visible.
RS4 — Comparación del MAC en tiempo constante	hasher.py:79–84 4-benchmark_timing.py:35–103	hmac.compare_digest; el benchmark no demuestra tiempo constante del endpoint completo.
Evidencia MitM	Servidor.py:71–86 evidencias/2mitm.pcapng	Alteración de cuerpo observada y rechazo 403.
Evidencia de red	evidencias/1normal.pcapng evidencias/2mitm.pcapng evidencias/3replay.pcapng evidencias/5logoutexit.pcapng	Capturas locales de loopback; redactar antes de distribución.

CONCLUSIÓN

Las evidencias recabadas confirman que los controles básicos del **SECBANK** operan de forma efectiva bajo los parámetros de las pruebas locales. El sistema logró procesar solicitudes legítimas y denegar accesos no autorizados, mitigando con éxito manipulaciones en el cuerpo de los mensajes (rechazo por HMAC inválido) y ataques de repetición (rechazo por duplicidad de nonce). Asimismo, la lógica de autenticación aplicó correctamente el bloqueo temporal tras intentos fallidos y la invalidación de sesiones (logout). A nivel de código, la implementación demuestra un uso adecuado de primitivas de seguridad esenciales: validación criptográfica (HMAC-SHA256), parámetros de un solo uso (nonces), control de frescura (ventanas temporales), derivación de contraseñas iterativa y mitigación de canales laterales mediante hmac.compare_digest().

Para la derivación de claves, se optó por implementar PBKDF2 con HMAC-SHA256 utilizando un salt aleatorio de 16 bytes y 600.000 iteraciones. Esta configuración cumple con los estándares actuales para mitigar ataques de fuerza bruta y diccionarios offline, incrementando el coste computacional por intento, lo que neutraliza la eficacia de hardware especializado (GPUs/ASICs). Por otro lado, para evitar la extracción de información a través de “side channels”(Timing Attacks), la validación de la firma MAC y las contraseñas emplea estrictamente hmac.compare_digest(). A diferencia de los comparadores de cadenas estándar que interrumpen la ejecución en el primer byte discordante (revelando fracciones de segundo medibles), esta función asegura un tiempo de ejecución constante analizando la longitud total del hash, ocultando al atacante cualquier pista sobre la validez parcial de la firma.

A pesar de estos aciertos técnicos, el prototipo presenta limitaciones estructurales que restringen su viabilidad a un ámbito estrictamente académico o de laboratorio. La dependencia de HTTP expone el tráfico a interceptación, comprometiendo la confidencialidad; el esquema de firma actual omite la validación del método, la ruta y las cookies de sesión; y la arquitectura carece de la validación transaccional y la gestión segura de claves indispensables en el sector financiero. En consecuencia, se dictamina que el sistema no es apto para entornos de producción ni debe procesar datos de clientes, credenciales o fondos reales bajo ninguna circunstancia.

CONFIDENTIAL

BIBLIOGRAFÍA

Universidad de Sevilla (2026). *PAI 1. IntegriDos - Verificadores de Integridad en la Transmisión para Entidad Financiera*. Documentación oficial del proyecto INSEGUS.

Python Software Foundation (2026). *Python 3 Documentation*. Referencias de las librerías criptográficas estándar: hmac, hashlib (SHA256), secrets (mitigación de timing attacks) y uuid.

Wireshark Foundation. *Wireshark Network Protocol Analyzer Documentation*. Guías de uso para inspección profunda de paquetes (DPI) y análisis de flujos HTTP/TCP en redes no cifradas.

OWASP Foundation (2026). Password Storage Cheat Sheet. Recomendaciones estándar de la industria para algoritmos de derivación de claves (PBKDF2, Argon2id) y factores de trabajo/iteraciones.

Google (2026). Gemini. Modelo de Lenguaje de Inteligencia Artificial. Utilizado como asistente de desarrollo y estructuración documental conforme a la normativa de la asignatura.

CONFIDENTIAL

8. ANEXO TÉCNICO

8.1. HERRAMIENTAS Y PROGRAMAS UTILIZADOS EN EL PENTEST

Durante la fase de desarrollo, auditoría y recolección de evidencias, el Security Team X empleó las siguientes herramientas e infraestructuras:

Entorno de Trabajo (OS): Kali Linux (Distribución orientada a la auditoría de seguridad y pruebas de penetración).

Desarrollo y Scripts de Ataque: Python 3 (Implementación del servidor API REST, el cliente legítimo y los scripts automatizados de inyección para simular ataques MitM y Replay).

Análisis de Tráfico de Red: Wireshark (Captura de interfaces locales, filtrado de tráfico en el puerto 8080, seguimiento de streams HTTP y exportación de trazas en formato .pcap).

Edición de Código: nano (Editor de texto en terminal para la modificación ágil de los scripts en Python).

Asistente de Inteligencia Artificial (IA): Gemini (Consultoría técnica, revisión de código seguro, explicación de vulnerabilidades y apoyo en la estructuración formal del informe técnico, en estricto cumplimiento de la normativa académica del proyecto).

8.2. TIEMPO DE CONCLUSIÓN DE LOS ANÁLISIS

- **Tiempo Total Estimado del Pentest Técnico:** 8 horas (incluyendo despliegue del entorno en Kali Linux, codificación de la capa de seguridad, captura de trazas en Wireshark y redacción de la documentación técnica).

8.3 LISTA DE PROMPTS UTILIZADOS (DECLARACIÓN DE USO DE IA)

1. De acuerdo con las directrices académicas sobre el uso ético y responsable de la Inteligencia Artificial, a continuación se detallan los *prompts* (instrucciones) más relevantes enviados al asistente de IA durante el proceso de comprensión teórica, desarrollo del código y estructuración del documento:
2. "Quiero que respondas a esta actividad como un equipo de consultoría para SecBank, por partes. Estructura de la documentación obligatoria..."
3. "Usar Wireshark" (Acompañado de capturas de pantalla de la interfaz).
4. "Simular el ataque Man-in-the-Middle (MitM): ¿qué debo observar?"
5. "Quiero elaborar el documento oficial; te enviaré todo y tú lo organizarás de manera profesional y ordenada. En la sección técnica, explica de forma didáctica lo que se realizó."
6. "Verificación de tiempo constante (mitigación de ataques de temporización): ¿no necesitaba probar esto?" (Validación de la mitigación de canales laterales).

CONFIDENTIAL

LEGAL NOTICE AND DISCLAIMER OF RESPONSIBILITY:

INSEGUS conducted this security assessment using methodologies aligned with internationally recognized standards and professional testing frameworks. The analysis was evidence-based and yielded reproducible results, prioritizing risks in accordance with standard practices for formal cybersecurity testing reports. The findings have been documented, detailed, and classified by severity and potential business impact. The recommendations herein should be treated as a priority action plan to mitigate legal, operational, and reputational risks to the organization.

This document contains highly sensitive and confidential technical information regarding critical vulnerabilities that could directly affect the organization's technology infrastructure and security model. Its disclosure, reproduction, or distribution—in whole or in part—without the express written authorization of security management is strictly prohibited.XXXX